

I decorator di Python

I decorators in Flask

Le funzioni sono **first class objects**:

- una funzione può essere passata come parametro di un'altra funzione, non il valore di ritorno della funzione ma la funzione stessa.

Quindi si può scrivere una cosa del genere:



Python

```
1 def fun1()->str:
2     return "Hello"
3
4 def fun2(fun):
5     fun()
6
7 fun2(fun1) #Da notare che passo la funzione stessa senza
             parentesi quello che viene indicato è la zona di memoria dove
             fun1 si trova
```

L'interprete py quando legge le funzioni prima di eseguirle va a leggere lo spazio in memoria dove sono salvate.

In questo caso sto passando il riferimento di memoria, se eseguo fun2 questa funzione richiama qualsiasi funzione gli sto passando in questo caso `fun()`.

Quindi quando eseguo fun2 questa esegue la funzione fun che altro non è che la funzione fun1 nell'invocazione di fun2.

In altre parole:



Python

```
1 def ciao()
2     return "Hello"
3
4 def fun2(fun):
5     fun()
6
7 fun2(ciao)
```

I decorators fanno qualcosa di simile: è una funzione che prende in input un'altra funzione, quella che vogliamo decorare, viene decorato dal decoratore, in questo caso fun2 sarebbe il decoratore mentre ciao è la funzione che vogliamo decorare.

Inner function e outer function

Una funzione può prendere in ingresso un'altra funzione ma può anche ritornarla: fun2 può prendere in ingresso ciao() ma può anche ritornare ciao().

Differenza tra funzioni esterne ed interne:

Le funzioni possono contenere quello che vogliamo:



Python

```
1 def parent():
2     print("Printing from parent()")
3
4     def first_child():
5         print("Printing from first child")
6     def second_child():
7         print("Printing from second child")
8     second_child()
9     first_child()
```

Quando la funzione parent è stata eseguita tutta se non trovo il modo di tirare fuori le funzioni interne non possono essere usate globalmente.

I decorators vanno ad espandere le funzionalità delle funzioni, l'esempio è il cronometro, usando un decorator sulla funzione del cronometro posso, ad esempio vedere il tempo di running della inner function cronometro().

Andiamo a vedere come fare un decorator cronometro:



Python

```
1 import time
2 def cronometro(fun): #è il decorator (@cronometro)
3     def wrapper() #avvolge la funzione che si vuole decorare
4         start = time.time()
5         fun()
6         print(time.time()-start)
7     return wrapper #non l'ho ritorno con le parentesi ma ritorno
il riferimento perché non voglio passare la funzione ma il suo
indirizzo di memoria
```

Una volta definito l'header ho bisogno di una inner function.

Ora:

voglio sapere quanto ci mette questa funzione a runnare, uso quindi il decorator cronometro



Python

```
1 @cronometro
2 def ciao():
3     print("hello")
```

Un modo per applicare il decoratore è così, quando scrivo la chiocciola sto facendo :



Python

```
1 ciao = cronometro(ciao)
```

Quindi quando assegno ciao al cronometro eseguo wrapper e i riferimenti di ciao si trovano nel corpo di wrapper (fun()) quindi la funzione ciao contiene il wrapper del cronometro.

Quindi se eseguo ciao() sto ritornando la funzione wrapper (), la differenza è che nella funzione wrapper fun() ha come valore ciao().

Questo



Python

```
1 ciao = cronometro(ciao)
```

viene definito da



Python

```
1 @cronometro
```

È solo sintassi, detto anche zucchero sintattico; se scriviamo `ciao = cronometro(ciao)` è la stessa cosa di `@cronometro`.

Quindi il decoratore è una funzione che prende in input una funzione e restituisce la funzione decorata.

Metto `*args` nel decorator perchè devo poter passar qualsiasi argomento nel wrapper e in fun().



Python

```
1  import time
2
3  def cronometro(fun):
4
5      def wrapper(*args):
6
7          start = time.time()
8
9          fun(*args)
10
11         print(time.time()- start)
12
13     return wrapper
14
15 @cronometro
16
17 def ciao():
18
19     print("Hello")
20
21 # ciao = cronometro(ciao)
22
23 # @cronometro
24
25 ciao()
```

Questo è un design pattern per risparmiare codice.